

Jev-like

Transformer: 基于共享状态与动态候选集的并行决策模型

摘要

传统大语言模型（LLM）主要采用自回归生成机制：给定上下文后逐 token 产生输出。当一个任务需要针对同一上下文回答大量相互独立的问题时，这种机制会产生大量重复计算。

本文提出一种 **Jev-like Transformer** 架构：将公共上下文表示为共享 State，将多个待回答问题表示为并行 Question Token，并通过 Transformer 一次计算得到每个问题的独立表示；随后利用候选答案（Candidate Answers）的 embedding 与问题表示进行匹配打分，形成长度可变的 logits，并通过 softmax 得到每个问题的概率分布。

该方法的核心特点是：**共享上下文计算、问题并行推理、候选答案动态决定输出空间，而无需为不同问题设计不同尺寸的输出 Head。**

1. 核心思想

传统 LLM:

```

State
  ↓
Transformer
  ↓
token1 → token2 → token3 → ... → answer
  
```

对于多个问题:

```

State → Question 1 → 生成答案
State → Question 2 → 生成答案
State → Question 3 → 生成答案
...
  
```

大量计算被重复执行。

本文采用:

```

          ┌→ question1 → answer1
          |
State ────┼─ Backbone ────┼→ question2 → answer2
          |
          └→ question3 → answer3
  
```

也就是:

一个共享 State，多个 Question 并行进入 Transformer，一次 forward 同时得到多个问题的答案。

2. 模型架构

设公共状态为:

$$S = (s_1, s_2, \dots, s_m)$$

有 N 个问题:

$$Q_i = (q_{i1}, q_{i2}, \dots, q_{iki})$$

模型首先对 State 进行编码:

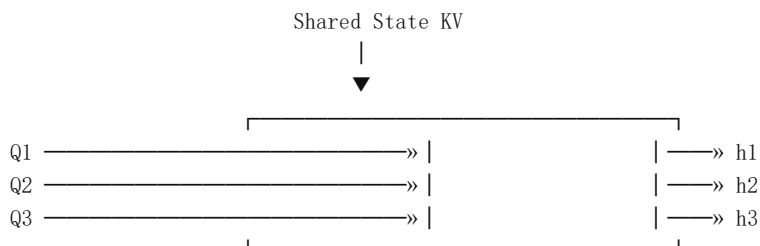
$$H_s = \text{Transformer}(S)$$

然后每个 Question 都可以访问公共 State, 同时保留自己的问题信息。

最终每个 Question 得到一个表示:

$$h_i \in \mathbb{R}^d$$

整体可以理解为:



其中:

- State: 所有问题共享的公共上下文;
- Question: 每个问题自己的 query token;
- h_i : 第 i 个问题经过 Transformer 后得到的语义向量。

3. Dynamic Candidate Head

这是该架构最关键的问题之一。

假设:

Q1: 是否退款?

答案 = {是, 否}

需要:

$\text{hidden} \rightarrow \text{Linear}(d, 2) \rightarrow \text{logits}[2]$

而:

Q2: 用户类型?

答案 = {普通用户, 高价值用户, 风险用户, 黑名单}

需要:

$\text{hidden} \rightarrow \text{Linear}(d, 4) \rightarrow \text{logits}[4]$

Q3 可能有 17 个候选:

$\text{hidden} \rightarrow \text{Linear}(d, 17) \rightarrow \text{logits}[17]$

因此:

不能简单地为每个 Question 使用一个固定尺寸的 FFN Head。

否则不同问题的答案数量不同, 就需要不同输出维度的 Head。

4. Candidate Embedding + Shared Scorer

解决方法是：

不让 Head 决定输出维度，而让 Candidate Set 决定输出维度。

对于每个候选答案 a ，得到一个 embedding：

$$e_a \in \mathbb{R}^d$$

问题表示：

$$h_i \in \mathbb{R}^d$$

使用一个共享 scorer 计算：

$$z_{ij} = f(h_i, e_{ij})$$

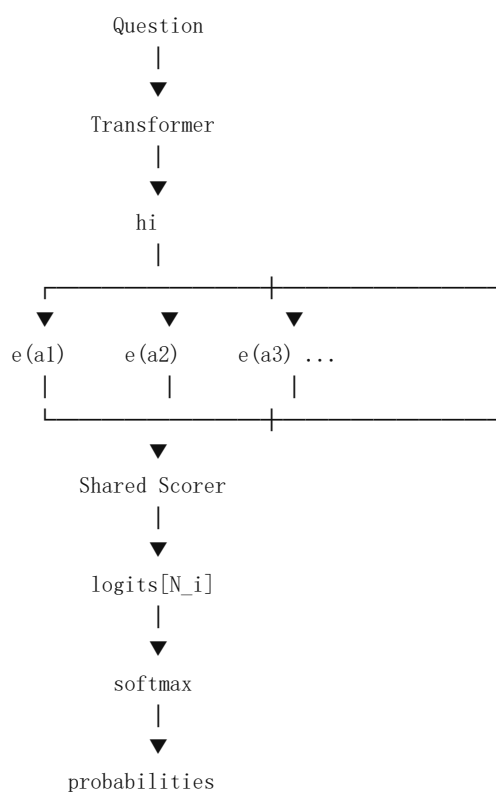
最简单的实现是点积：

$$z_{ij} = h_i \cdot e_{ij} / \sqrt{d}$$

然后：

$$P(a_{ij} | S, Q_i) = \text{softmax}_j(z_{ij})$$

完整结构：



这样：

Question 1:

Candidates = [Yes, No]

↓

logits = [2]

Question 2:

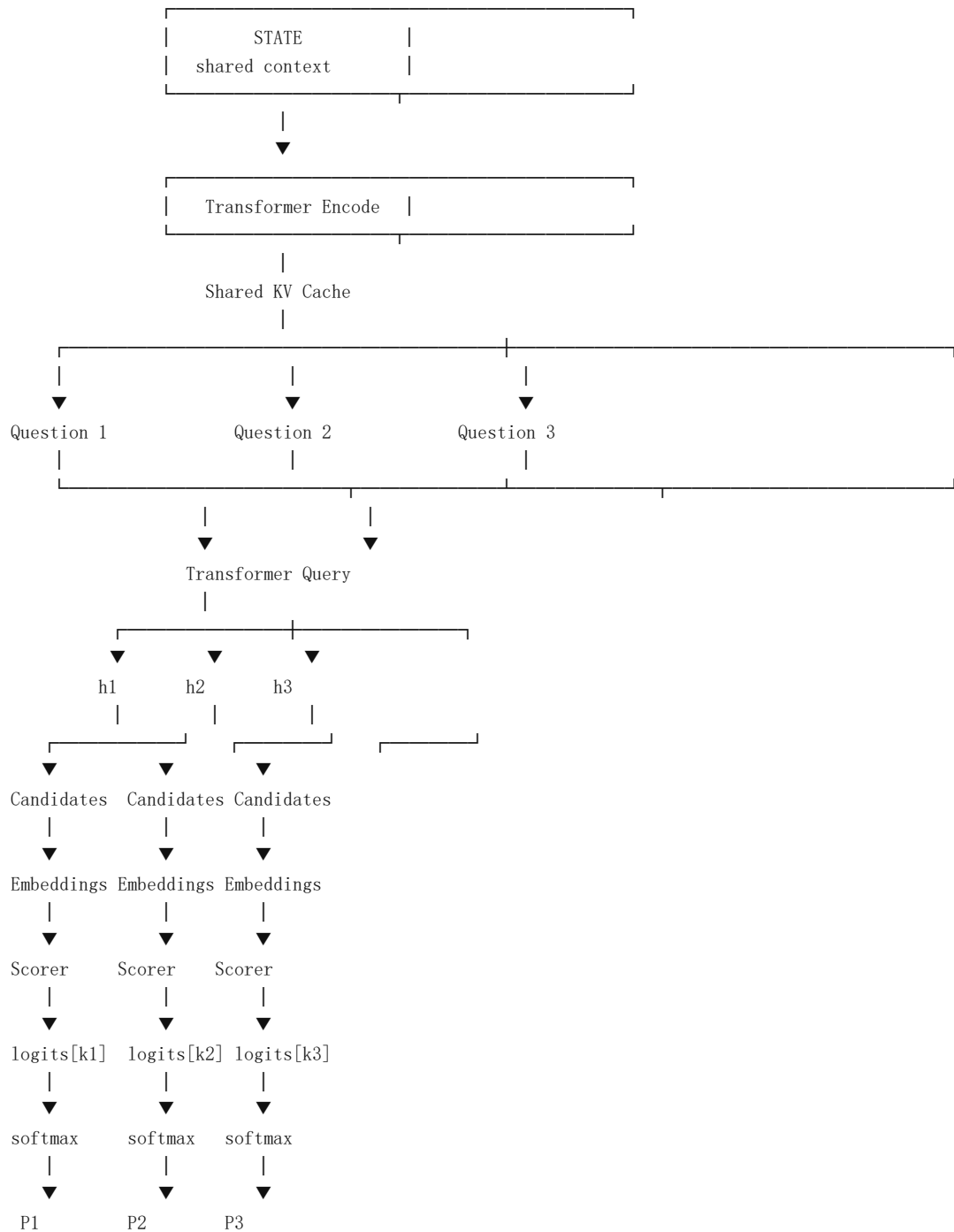
Candidates = [A, B, C, D, E]

↓
logits = [5]

Question 3:
Candidates = [A1 ... A100]
↓
logits = [100]

模型本身不需要改变。

5. 完整架构



核心公式可以浓缩为：

$$P(a_{ij} | S, Q_i) = \text{softmax}_j(f(h_i, e_{ij}))$$

其中:

- S: 共享 State;
- Q_i : 第 i 个 Question;
- h_i : Question 表示;
- a_{ij} : 第 i 个问题的第 j 个候选答案;
- e_{ij} : 候选答案 embedding;
- z_{ij} : 候选答案 logit。

6. 为什么这个结构重要

传统 LLM 的输出空间固定为整个 vocabulary:

$$V \approx 10^4 \sim 10^5$$

而 Jev-like 模型的输出空间变成:

$$A_i = \{a_{i1}, a_{i2}, \dots, a_{iki}\}$$

即:

每个问题拥有自己的动态输出空间。

这使模型天然适合:

- 分类;
- 路由;
- Agent 决策;
- Tool selection;
- 风险判断;
- 意图识别;
- 多条件判断;
- Ranking;
- 结构化决策。

尤其适合:

一个 State

+

大量独立 Questions

+

每个 Question 不同数量、不同内容的候选答案

7. 工程实现

7.1 第一版不需要修改 Transformer

最容易落地的方法是直接基于现有 HuggingFace Transformer。

输入:

State tokens
+
Question tokens

通过 attention mask 控制:

Question → 可以看到 State
Question → 可以看到自己
Question → 不需要看到其他 Question

例如:

	State	Q1	Q2	Q3
State	✓	✓	✓	✓
Q1	✓	✓	×	×
Q2	✓	×	✓	×
Q3	✓	×	×	✓

这样就能在一次 Transformer forward 中完成多个 Question 的计算。

7.2 第一版最简单实现

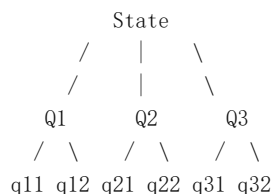
可以直接把:

State + Question

组织成:

[STATE TOKENS] [QUESTION TOKENS]

然后构造 attention mask:



Q1、Q2、Q3 可以并行 batch。

8. Candidate Scorer 的 PyTorch 实现

最简单版本甚至不需要额外 FFN:

```
class CandidateScorer(nn.Module):
    def forward(self, h, candidate_embeddings):
        # h: [B, D]
        # candidate_embeddings: [B, K, D]

        logits = torch.einsum(
            "bd, bkd->bk",
            h,
            candidate_embeddings
        )

        return logits
```

然后:

```
logits = scorer(h, candidates)

probs = torch.softmax(logits, dim=-1)
```

这里 K 可以不同。

实际 batch 中可以 padding:

```
Q1: K=2
Q2: K=5
Q3: K=17
```

↓

```
[Q1: 2 valid + padding]
[Q2: 5 valid]
[Q3: 17 valid]
```

通过 candidate mask:

```
logits = logits.masked_fill(~candidate_mask, -inf)
```

即可。

9. 更强的 Scorer

点积只是最简单实现。

可以使用:

$$z = f([h; e])$$

例如:

```
h -----
      |
      |----- concat -----> MLP -----> scalar
e -----
```

或者:

$$z = h^T W e$$

即双线性 scorer:

```
logit = h @ W @ e
```

还可以:

```
h
|
|----- MLP -----> query embedding
|
e
|
|----- MLP -----> candidate embedding
```

```
query embedding • candidate embedding
```

这样 Question 和 Candidate 可以使用不同的 projection。

10. Candidate 的来源

Candidate 不一定需要固定存储。

可以有三种模式。

模式 A: 固定 Candidate Vocabulary

例如:

```
["yes", "no"]
```

直接维护 embedding table。

模式 B: 自然语言 Candidate

例如:

Question:

“客户应该采取什么操作？”

Candidates:

```
[  
  "退款",  
  "换货",  
  "拒绝",  
  "人工审核"  
]
```

把每个 candidate 文本送入 Candidate Encoder:

```
"退款" → embedding  
"换货" → embedding  
"拒绝" → embedding  
"人工审核" → embedding
```

然后进行 scoring。

模式 C: 运行时动态 Candidate

Agent 场景尤其适合:

State

+

Question:

“下一步调用哪个工具？”

Candidates:

```
[  
  search_web,  
  database,  
  calculator,  
  send_email,  
  browser  
]
```

Candidate Set 在运行时产生。

模型只需要对这些候选进行 scoring。

11. 训练方法

训练数据可以表示成:

State
Question
Candidates
Correct Answer

例如:

State:
客户购买手机后要求退款……

Question:
是否应该退款?

Candidates:
["yes", "no"]

Label:
yes

训练目标:

$$L = -\log P(a^* | S, Q)$$

多个 Question 可以组成一个 State 下的训练样本:

State S

Q1 → A1
Q2 → A2
Q3 → A3
...
QN → AN

总 loss:

$$L = \sum_i L_i$$

这样模型天然学习:

同一个 State 下, 同时解决多个独立决策问题。

12. 与传统 LLM 的根本区别

	LLM	Jev-like Transformer
输出单位	Token	Candidate
输出空间	固定 Vocabulary	Question-specific
推理方式	自回归	并行决策
多问题	重复生成	Batch 并行
State	每次参与生成	可共享 KV
输出长度	不固定 token 序列	Candidate 数量

	LLM	Jev-like Transformer
主要目标	生成文本	决策/分类/选择

因此它不是简单的：

“给 LLM 加几个分类 Head。”

更准确地说，它改变了模型的**输出范式**：

Token Generation -> Candidate Scoring

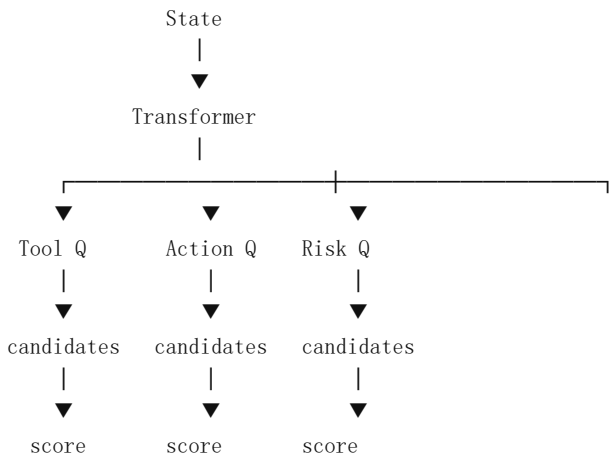
13. 一个值得进一步研究的方向

该架构实际上可以继续向：

State + Questions + Candidate Actions

发展。

例如 Agent：



最终模型不再负责生成：

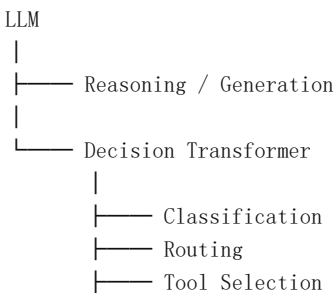
“我认为下一步应该调用搜索工具，因为……”

而直接输出：

search_web	0.71
database	0.18
calculator	0.07
browser	0.04

语言模型负责理解 State，Decision Model 负责进行大量结构化决策。

这可能形成一种新的模型分工：



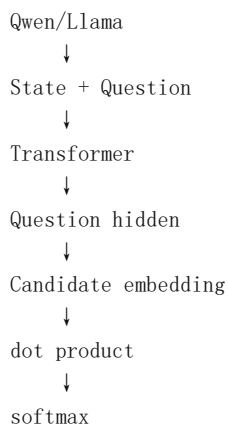


14. 最小可行系统

第一版建议不要直接修改 FlashAttention、PagedAttention 或底层 Transformer。

按照以下路线实现：

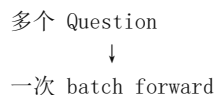
V0



先证明模型能够完成任务。

V1

加入：



验证吞吐提升。

V2

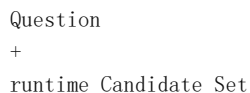
加入：

State KV Cache

让多个 Question 共享 State KV。

V3

Candidate 改成动态输入：



V4

针对推理优化：

Paged KV Cache
FlashAttention
Continuous Batching
Candidate Embedding Cache

Tensor Parallel
GPU fused scorer

最终形成真正面向 Agent Decision Loop 的高速 Decision Model。

15. 核心贡献

本文架构可以归纳为四个核心设计：

① Shared State

同一个上下文只需要进行一次公共计算。

② Parallel Questions

多个 Question 在同一次 Transformer 推理中并行处理。

③ Dynamic Candidate Set

每个 Question 拥有自己的 Candidate Set，输出维度动态确定。

④ Shared Candidate Scorer

不为不同 Candidate 数量设计不同 Prediction Head，而是通过 Candidate Embedding + Shared Scorer 统一处理。

最终模型可以表达为：

$$S + \{Q_1, \dots, Q_n\} \rightarrow \{h_1, \dots, h_n\} \rightarrow \text{Candidate Scoring} \rightarrow \{P_1, \dots, P_n\}$$

其中每个：

$$P_i = \text{softmax}(f(h_i, E_i))$$

而：

$$|E_1|, |E_2|, \dots, |E_n|$$

完全可以不同。

16. 结论

Jev-like Transformer 的核心不是简单增加多个分类 Head，而是重新定义 Transformer 的输出方式：

Autoregressive Language Generation $\rightarrow$ Parallel Candidate-based Decision

其最小可实现结构为：

Shared State + Parallel Questions + Dynamic Candidates + Shared Scorer

其中最关键的设计是：

Head 不再决定输出维度，Candidate Set 决定输出维度。

这使一个统一 Transformer 可以在同一个 State

下同时处理不同数量、不同内容的候选答案，并天然适用于分类、路由、Tool Selection、Ranking、Risk Assessment 以及 Agent Action Selection 等任务。

从工程角度，该方案可以直接建立在现有 Qwen/Llama Transformer 之上，不要求重新设计 Transformer Backbone；第一版只需要实现 **Attention Mask**、**Question Batch**、**Candidate Embedding** 和 **Shared Scorer**，即可构建一个完整的 Jev-like 原型。